

Week 4: Cleaning and Shaping Data

AI Frameworks

Today's learning goals

By the end of this lecture you should be able to:

1. Explain what missing values mean and choose simple drop vs fill strategies
2. Remove duplicates and rename columns
3. Sort and use `groupby` for category summaries
4. Describe a merge/join at a high level

Agenda (80 minutes)

1. Why cleaning matters
2. Missing values
3. Duplicates and renaming
4. Sorting and groupby
5. Merging tables (idea)

Real data is messy

- Empty cells, typos in categories, wrong types
- Duplicate rows from copy-paste or bad exports
- Two tables that should be combined

Cleaning is not glamorous—but it decides whether your later plots/models make sense.

What is “missing”?

- No value recorded: blank, NaN, sometimes None
- Reasons: not collected, not applicable, sensor failure, privacy, ...

First question: **how many** and **in which columns**?

Spotting NaNs

```
1 df.isna().sum()           # missing count per column
2 df["score"].isna().mean() # fraction missing
```

Drop vs fill (intuition)

Drop

Remove rows (or columns) with missing values

OK if few missing and random

Fill

Replace with a typical value (mean/median/mode) or a constant

OK as a simple start—note what you did

No universal best answer—document your choice.

Simple code patterns

```
1 df2 = df.dropna()           # drop rows with any NaN
2 df3 = df.dropna(subset=["score"]) # only if score missing
3
4 df4 = df.copy()
5 df4["score"] = df4["score"].fillna(df4["score"].median())
```

Quiz 1 — Missing data

Quiz

True or false?

Dropping all rows with any missing value is always the best approach.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

False. If many rows have gaps, dropping can throw away most of the data. Sometimes fill (or drop only critical columns) is wiser.

Duplicates

```
1 df.duplicated().sum()
2 df = df.drop_duplicates()
3
4 # optional: based on some columns only
5 df = df.drop_duplicates(subset=["id"])
```

Renaming columns

```
1 df = df.rename(columns={  
2     "Score": "score",  
3     "Student Name": "name"  
4 })  
5 # prefer clear, consistent, snake_case-ish names
```

Common mistake

Leaving spaces and mixed caps in names—annoying later in code.

Sorting

```
1 df.sort_values("score")  
2 df.sort_values("score", ascending=False)  
3 df.sort_values(["year", "score"])
```

groupby: summarise by category

Idea: split rows into groups (e.g. by year or genre), then compute a summary (mean score, count, ...). Question example: "What is the average rating per genre?"

groupby examples

```
1 df.groupby("year")["score"].mean()  
2 df.groupby("genre")["rating"].agg(["count", "mean"])
```

Quiz 2 — groupby

Quiz

You want the average temp for each city. Which matches the idea?

1. `df.groupby("city")["temp"].mean()`
2. `df["temp"].head()`
3. `df.shape`
4. `df.dropna()`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

1. Group by city, then average the temp column.

Join idea (high level)

Two tables share a key (e.g. `student_id`):

- Table A: id, name
- Table B: id, score
- Merge → one table with name + score

Like a VLOOKUP / spreadsheet lookup—pandas calls it `merge`.

merge sketch

```
1 combined = pd.merge(students, scores, on="id", how="left")  
2 # how="inner" keeps only matching keys  
3 # how="left" keeps all rows from the left table
```

Details of join types can wait—remember: need a shared key.

Quiz 3 — Merge

Quiz

To combine two tables on a shared `id` column, you typically use:

1. `pd.merge(...)`
2. `df.head()`
3. `matplotlib`
4. `print(id)`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 3

Answer

1. `pd.merge` (joining on a key such as `id`).

Key takeaways

1. Find missingness; drop or fill with a noted strategy
2. Deduplicate; rename for clarity
3. `groupby` answers “per category” questions
4. `merge` combines tables via a shared key

Next up

Practice: clean a messy CSV + one groupby observation.
Week 5: visualisation with matplotlib (and a little seaborn).